

# Esercizio finale

## Generazione dei dati di input

Consideriamo un vettore di  $n$  elementi,  $n=10$ , in cui ogni elemento è una struttura

```
{int v1; int v2; int v3}
```

Il vettore viene inizializzato con i seguenti valori per i campi della struttura:

`v1` assume i valori 7, 1, 2, 0, 9, 6, 4, 5, 8, 3

`v2`, `v3` assumono un numero casuale tra 1 e 100, imponendo il seed 42 in modo che ogni volta `v2` e `v3` siano deterministici e l'esperimento sia replicabile.

---

1) creare con allocazione dinamica un vettore `v1` con le seguenti caratteristiche

- ogni elemento contiene un puntatore alla struttura `Elemento`, non direttamente un `Elemento`
- il puntatore punta ad un *nuovo* elemento, i cui dati sono letti da `vettore_input`
- Stampare gli elementi di `v1` mediante la funzione `printArray`

La `generaVettoreInput` simula i dati che possono essere prodotti dal mondo reale o dall'utente. Il programma prende questi dati e li inserisce nel vettore `v1`.

Se non si riesce a fare questo passo , si può procedere con il vettore di partenza.

---

2) Scrivere e applicare mergesort che ordini `v1` mediante il campo `v1`

l'ordinamento non copia le strutture: riordina i puntatori contenuti nel vettore `v1`

Se non si riesce ad implementare mergesort, si applichi un altro algoritmo

- 
- 3) mostrare che mediante strategy design pattern è possibile ordinare secondo altri campi, mantenendo il disaccoppiamento.

Creare una seconda funzione `mergesort_strategy` che accetta come quarto parametro il puntatore a funzione che implementa la strategia di ordinamento concreta

Implementate le seguenti strategie:

- ordina rispetto a `v2`
- ordina rispetto a `v1+v2+v3`

Richiamare la `mergesort_strategy` prima con `gt_v2` e poi con `gt_somma`. Dopo ogni ordinamento stampare il risultato

- 
- 4) Scrivere una funzione di ricerca binaria adatta a questo tipo di input. Utilizzarla per effettuare una ricerca dell'elemento con `v2 = 51`

- 
- 5) Implementare tutte le funzioni atte a costruire un MIN -HEAP. Create un min heap a partire da `v1` non ordinato. Ovviamente occorre specificare il criterio d'ordine usato dal min heap.

Il MIN HEAP può contenere direttamente i dati di `ELEMENTO`, non i puntatori. Non è una scelta corretta (meglio maneggiare i puntatori), ma è una scelta didattica)

- 
- 6) creare un ADT queue che incapsuli i dati di `v1`. Creare queue a partire da `v1` non ordinato. Anche qui queue contiene direttamente i dati di `ELEMENTO`, non i puntatori.